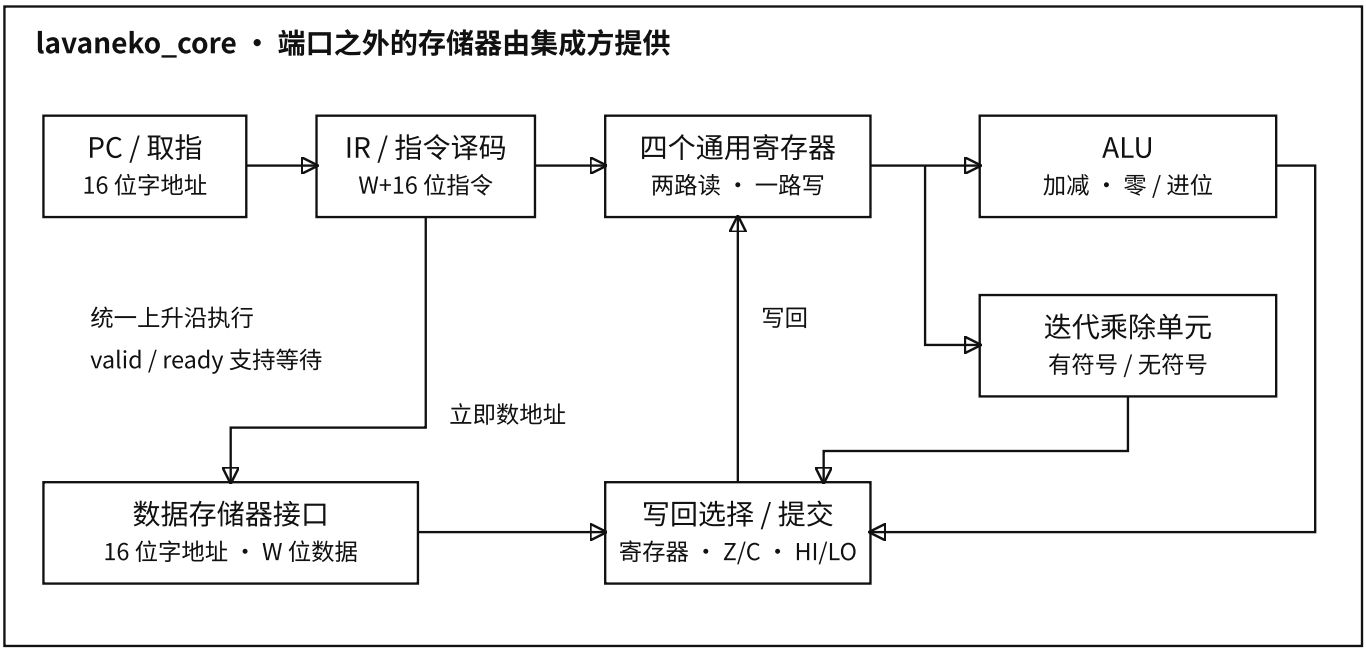
milku

Lavaneko 核心



组成	当前实现
数据 / 指令	数据 W=16 或 32 位；每条指令 W+16 位
通用寄存器	REG1..REG4，各 W 位；两路读、一路写，没有常零寄存器
PC / 存储器	16 位字地址；程序与数据接口独立；一次处理一条指令
算术	整数加减、比较、迭代乘除；结果截断至规定宽度
状态	Z 零标志、C 进位 / 无借位；HI、LO；MD_ERROR
时钟 / 复位	统一上升沿；同步高有效复位；核心不负责清空外部 RAM

执行路径

取指 → 译码与读寄存器 → 运算 / 访存 → 写回与更新 PC

ALU 是组合加减电路；乘除单元需要多拍。数据访问通过 valid / ready 等待完成。图中的模块不是流水线级数。

实现范围

现有 18 条命名指令的 Verilog 移植。尚无 SIMD、浮点、缓存、纹理或光栅化单元，不能直接运行 Unity。没有加入新的 GPU 指令。

源码：rtl/lavaneko_core.v、lavaneko_regfile.v、lavaneko_alu.v、lavaneko_muldiv.v

milku

一条指令怎么拆开

W+15 : 16	15 : 0
立即数 · W 位	控制字段 · 16 位

15:0 表示从第 15 位到第 0 位，包含两端，共 16 位。位号从右向左递增，不是比例。

控制字段位置	普通指令含义
0	减法选择
1	写通用寄存器
2	选择 RAM 读值写回
3	写 RAM
4	无条件跳转
5	B 操作数置零
6	HALT
7	A 选择寄存器，为 0 时选择立即数
9:8	B 寄存器编号
11:10	目标寄存器编号
13:12	A 寄存器编号
14	写 Z/C
15	按旧 Z 条件跳转

扩展优先：bit7:6 为 11 时，低八位是乘除扩展操作码，不能再按普通控制开关解释。13:12、11:10、9:8 仍是寄存器字段。

REG1..REG4 的编码依次为 `00 / 01 / 10 / 11`。RAM 与跳转使用立即数的低 16 位；PC 每加 1 指向下一整条指令。

32 位数据版本的指令长为 48 位，并非 32 位。例：`00000001 0022`，左侧 32 位表示常数 1，右侧 16 位表示写入 REG1。

普通字段可组合；命名指令采用下一页的标准组合。编码仍沿用 Lavaneko，不是 RISC-V 编码。

milku

全部命名指令

`aa / bb / dd` 是两位寄存器编号；`--` 是不参与该指令结果的字段，建议编码为 00。表中只列低 16 位；高 W 位为立即数，不使用时填零。

指令	低 16 位二进制	操作	Z/C
`NOP`	`00 -- -- -- 0000 0000`	无状态写入，PC+1	保持
`LDI Rd, imm`	`00 -- dd -- 0010 0010`	Rd ← imm	保持
`ADDI Rd, Rb, imm`	`01 -- dd bb 0000 0010`	Rd ← imm + Rb	更新
`ADD Rd, Ra, Rb`	`01 aa dd bb 1000 0010`	Rd ← Ra + Rb	更新
`SUB Rd, Ra, Rb`	`01 aa dd bb 1000 0011`	Rd ← Ra − Rb	更新
`CMP Ra, Rb`	`01 aa -- bb 1000 0001`	计算 Ra − Rb，不写寄存器	更新
`LOAD Rd, addr`	`00 -- dd -- 0000 0110`	Rd ← RAM[addr]	保持
`STORE Rb, addr`	`00 -- -- bb 0000 1000`	RAM[addr] ← Rb	保持
`JUMP addr`	`00 -- -- -- 0001 0000`	PC ← addr	保持
`JZ addr`	`10 -- -- -- 0000 0000`	旧 Z=1 时跳转，否则 PC+1	保持
`HALT`	`00 -- -- -- 0100 0000`	停止；PC 保持	保持
`MULU Rd, Ra, Rb`	`00 aa dd bb 1100 0000`	无符号乘积 → HI:LO；Rd ← LO	保持
`MUL Rd, Ra, Rb`	`00 aa dd bb 1100 0001`	有符号乘积 → HI:LO；Rd ← LO	保持
`DIVU Rd, Ra, Rb`	`00 aa dd bb 1100 0010`	无符号商 → LO，余数 → HI	保持
`DIV Rd, Ra, Rb`	`00 aa dd bb 1100 0011`	有符号商 → LO，余数 → HI	保持
`MFLO Rd`	`00 -- dd -- 1100 0100`	Rd ← LO	保持
`MFHI Rd`	`00 -- dd -- 1100 0101`	Rd ← HI	保持
`MDSTAT Rd`	`00 -- dd -- 1100 0110`	Rd ← MD_ERROR（零扩展）	保持

DIV/DIVU 成功时 Rd ← LO；有符号除法朝零截断，余数符号与被除数相同。乘除扩展忽略 bit15:14 和立即数，建议填零。

MD_ERROR	含义	结果
0	成功	更新 HI/LO 和目标寄存器
1	除数为零	HI=LO=0；目标寄存器保持；PC+1
2	有符号最小值除以 -1	HI=LO=0；目标寄存器保持；PC+1

减法 C=1 表示没有借位；没有独立有符号溢出标志。扩展码 C7..FF 为保留 NOP；不触发 RAM、分支、HALT 或标志更新。

汇编写法用于说明语义；当前工程没有文本汇编器。测试机器字由上游编码接口生成。

milku

从 1 + 1 开始

PC	指令	32 位数据版本的 48 位机器字
0	LDI REG1, 1	`000000010022`
1	LDI REG2, 1	`000000010422`
2	ADD REG3, REG1, REG2	`000000004982`
3	HALT	`000000000040`

写入常数 1

PC=0 → 请求程序存储器 → IR 收到 000000010022
译码：A 选择立即数 1，B 选择 0，打开 REG1 写使能
ALU：1+0=1 → 写回 REG1 → PC=1

第二条相同，只把目标寄存器改为 REG2。LDI 不改变 Z/C。

相加得到 2

PC=2 → IR 收到 000000004982
译码：Ra=REG1，Rb=REG2，Rd=REG3
寄存器组读出 1 和 1 → ALU 算出 2 → 写回 REG3
更新 Z=0、C=0 → PC=3 → 执行 HALT 后保持 PC=3

时序

指令	无等待时的核心拍数
普通运算 / 跳转 / HALT	2：取指、执行提交
LOAD / STORE	3：取指、执行、存储器握手提交
正常乘除	W+4：取指、单元运算、提交
错误除法	4；错误也作为完成的指令提交

外部 ready 为 0 会增加等待拍数。复位清零核心，取消未完成请求；外部存储器须配合复位。已经完成的内存写入不会被撤销。

验证状态

16/32 位各 3,976 次参考提交，分别运行有等待与无等待；乘除单元每种位宽独立检查 659 次。另验证复位、背压和 PC 回绕。通用综合通过；未执行 Vivado 布局布线或上板测试。

基线：lavamilkTeam/lavanekoCPU · 8e249ae491903985db7ca79df683efa5b7946267
详细接口与复现命令：interfaces.md、verification.md。文档日期：2026-10-22。